

PRACTICAL- 6 (A)

AIM

Constraint Satisfaction

- a. Write a python code for implementing Cryptarithmic problem.

THEORY

A Cryptarithmic problem is a type of Constraint Satisfaction Problem (CSP) in Artificial Intelligence where letters in an arithmetic equation are replaced by digits. The objective is to find the correct digit for each letter so that the arithmetic expression becomes valid.

In a cryptarithmic puzzle, each letter represents a unique digit from 0 to 9, and no two letters can have the same digit. Additionally, the first letter of a number cannot be assigned the digit 0, because numbers cannot start with zero.

One of the most common examples of a cryptarithmic problem is:

SEND + MORE = MONEY

In this puzzle, each letter (S, E, N, D, M, O, R, Y) represents a different digit. The task is to determine the digit assigned to each letter such that the addition is mathematically correct.

To solve this problem using Python, the program generates different combinations of digits for the letters and checks whether the arithmetic condition is satisfied. The program ensures that each letter has a unique digit and verifies the equation after assigning the digits.

Cryptarithmic problems are useful for demonstrating the concept of constraint satisfaction, where the solution must satisfy multiple constraints simultaneously. They are widely used in Artificial Intelligence to illustrate search techniques, logical reasoning, and problem-solving strategies.

SOFTWARE USED

Jupyter Notebook

CODE

```
from itertools import permutations
```

```
letters = ('S','E','N','D','M','O','R','Y')
```

```
digits = range(10)
```

```
for perm in permutations(digits, len(letters)):
```

```
    s,e,n,d,m,o,r,y = perm
```

```
    if s == 0 or m == 0:
```

```
        continue
```

```
    send = 1000*s + 100*e + 10*n + d
```

```
    more = 1000*m + 100*o + 10*r + e
```

```
    money = 10000*m + 1000*o + 100*n + 10*e + y
```

```
    if send + more == money:
```

```
        print("Solution Found: Cryptarithmic problem. ")
```

```
        print("SEND =", send)
```

```
        print("MORE =", more)
```

```
        print("MONEY =", money)
```

```
        print("\nLetter Mapping:")
```

```
        print("S =", s)
```

```
        print("E =", e)
```

```
        print("N =", n)
```

```
        print("D =", d)
```

```
        print("M =", m)
```

```
        print("O =", o)
```

```
        print("R =", r)
```

```
        print("Y =", y)
```

```
    break
```

OUTPUT

Solution Found: Cryptarithmic problem.

SEND = 9567

MORE = 1085

MONEY = 10652

Letter Mapping:

S = 9

E = 5

N = 6

D = 7

M = 1

O = 0

R = 8

Y = 2

PRACTICAL- 6 (B)

AIM

Constraint Satisfaction

b. Write a python code for implementing Graph colouring problem.

THEORY

The Graph Colouring Problem is a well-known Constraint Satisfaction Problem (CSP) in Artificial Intelligence and graph theory. The objective of this problem is to assign colors to the vertices of a graph in such a way that no two adjacent vertices have the same color.

In a graph, vertices represent nodes and edges represent the connections between those nodes. If two vertices are connected by an edge, they are called adjacent vertices, and they must be assigned different colors.

The main goal of the graph colouring problem is to determine whether it is possible to color the graph using a limited number of colors while satisfying the given constraints. The minimum number of colors required to color a graph without violating the rules is known as the chromatic number of the graph.

In Artificial Intelligence, this problem is often solved using backtracking or constraint satisfaction techniques. The algorithm assigns colors to vertices one by one and checks whether the assignment satisfies the constraints. If a conflict occurs, the algorithm backtracks and tries another color.

Graph colouring has many practical applications such as map colouring, scheduling problems, register allocation in compilers, and frequency assignment in wireless networks. It helps ensure that conflicting elements do not share the same resource.

SOFTWARE USED

Jupyter Notebook

CODE

```
def is_safe(v, graph, color, c):  
    for i in graph[v]:  
        if color[i] == c:  
            return False  
    return True
```

```

def solve_graph_coloring(graph, m, color, v, n):

    if v == n:
        return True

    for c in range(1, m + 1):

        if is_safe(v, graph, color, c):
            color[v] = c

            if solve_graph_coloring(graph, m, color, v + 1, n):
                return True

            color[v] = 0

    return False


n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))

graph = {i: [] for i in range(n)}

print("Enter edges (vertex1 vertex2):")

for _ in range(e):
    u, v = map(int, input().split())
    graph[u].append(v)

```

```

graph[v].append(u)

m = int(input("Enter number of colors: "))

color = [0] * n

if solve_graph_coloring(graph, m, color, 0, n):

    print("\nColor Assignment:")
    for i in range(n):
        print("Vertex", i, "-> Color", color[i])
else:
    print("No solution exists with given number of colors")

```

OUTPUT

```

Enter number of vertices: 4
Enter number of edges: 4
Enter edges (vertex1 vertex2):
0 1
0 2
1 2
1 3
Enter number of colors: 3

Color Assignment:
Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 3
Vertex 3 -> Color 1

```
